



Unidad 4b : Mapas interactivos

Introducción a Folium

Mapas

Elementos de Interfaz de Usuario

Capas Raster

Capas Vectoriales

Capas GeoJSON y TopoJSON

Mapas de coropletas

Funciones de estilo

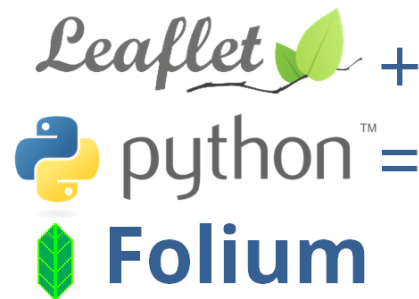
Integración GeoPandas-Folium

Extensiones

Visualización de ficheros GPX

Fuentes de datos

- ▶ Folium es una librería que utiliza Python para manipular los datos y leaflet.js para dibujar mapas
- ▶ Puede trabajar con OpenStreetMap y Mapbox
- ▶ El resultado es un fichero html



Introducción a Folium

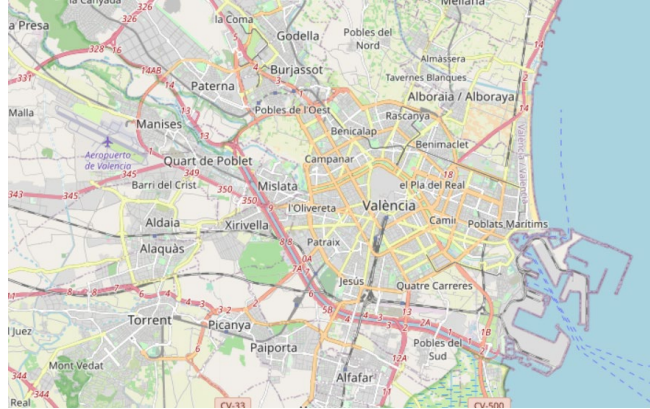


- ▶ Instalación:
 - ▶ pip install folium
- ▶ Importación:
 - ▶ import folium

- ▶ Crear un mapa:

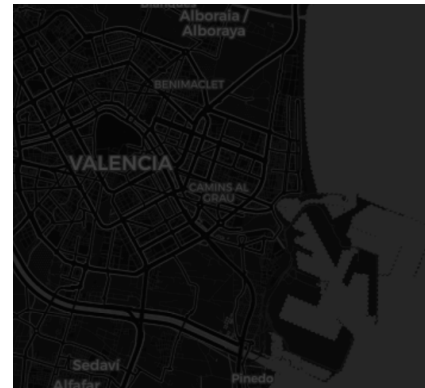
```
m = folium.Map(location=[39.526500, -1.143250])  
m.save("index.html")
```

- ▶ Incluir zoom inicial: zoom_start=12
- ▶ Por defecto utiliza las teselas de OpenStreetMap, en la actualidad da error (utilizar otros servidores)



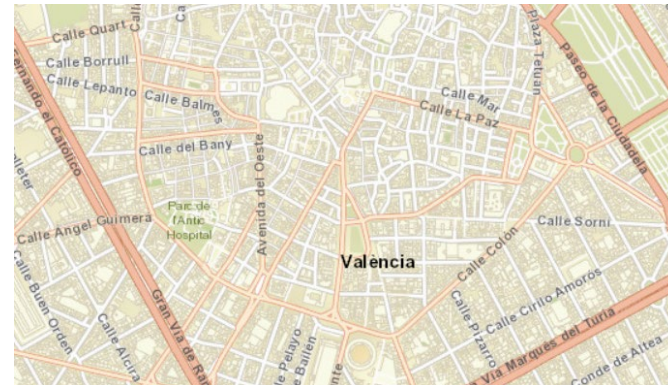
```
m = folium.Map(location=[39.526500, -  
1.143250], tiles="CartoDB positron")
```

Otro: CartoDB dark_matter



- ▶ Se pueden utilizar los proveedores de mapas de Leaflet:
 - ▶ <http://leaflet-extras.github.io/leaflet-providers/preview/>

```
m = folium.Map(location=[39.526500, -1.143250],  
tiles="https://server.arcgisonline.com/ArcGIS/rest/services/World_Street_Map/MapServer/tile/{z}/{y}/{x}",  
attr='Tiles &copy; Esri &mdash; Source: Esri, DeLorme, NAVTEQ, USGS, Intermap, iPC, NRCAN, Esri Japan,  
METI, Esri China (Hong Kong), Esri (Thailand), TomTom, 2012')
```



- ▶ Controles: Escala, Zoom y Capas
 - ▶ Escala: Muestra la escala en la parte inf-izq
 - ▶ Zoom: Muestra el control de zoom en el parte sup-izq
 - ▶ Capas: Muestra el control de capas en el parte sup-der

```
m = folium.Map(location=[39.526500, -1.143250], control_scale=True,  
zoom_control=True)  
folium.TileLayer(tiles=..., opacity=...)  
folium.LayerControl().add_to(m)
```



- ▶ Se pueden incluir límites para que no se pueda realizar desplazamientos fuera de ellos:

```
min_lon, max_lon = -45, -35
min_lat, max_lat = -25, -15
m = folium.Map(
    max_bounds=True,
    location=[-20, -40],
    zoom_start=6,
    min_lat=min_lat,
    max_lat=max_lat,
    min_lon=min_lon,
    max_lon=max_lon)
```



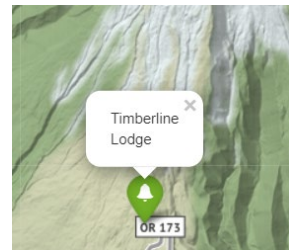
- ▶ Además de los controles vistos, tenemos: marcas desplegadas e iconos
- ▶ La más sencilla es una marca de localización con información

```
tooltip = "Clica aquí!"
```

```
folium.Marker([45.3288, -121.6625], popup="<i>Mt. Hood  
Meadows</i>", tooltip=tooltip).add_to(m)
```

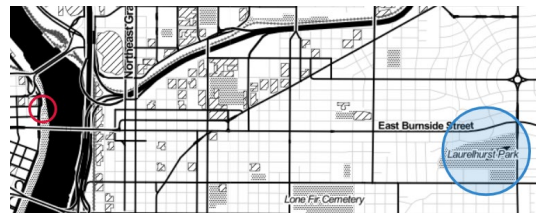


- ▶ Se pueden incluir círculos:
 - ▶ Circle: radio en metros
 - ▶ CircleMarker: radio en píxeles



```
folium.Circle(radius=100,location=[45.5244, -122.6699],popup="The Waterfront",  
color="crimson", fill=False).add_to(m)
```

```
folium.CircleMarker(location=[45.5215, -122.6261],radius=50,popup="Laurelhurst Park",  
color="#3186cc",fill=True,fill_color="#3186cc").add_to(m)
```



► Marcas con código html:

```
html = """
```

```
<h1> Esta es una marca emergente grande </h1><br>
```

```
Con unas pocas líneas de código, escrito en html...
```

```
<p><code>
```

```
    from numpy import *<br>
```

```
    exp(-2*pi)
```

```
</code></p>
```

```
"""
```

```
folium.Marker([45.3288, -121.6625], popup=html, tooltip=tooltip).add_to(m)
```

► Se puede añadir un marco:

```
iframe = branca.element.IFrame(html=html, width=500, height=300)
```

```
popup = folium.Popup(iframe, max_width=500)
```



- ▶ La función `LatLngPopup()` informa sobre longitud y latitud del punto en el que se pincha
`m.add_child(folium.LatLngPopup())`
- ▶ Se pueden incluir marcas de forma interactiva:
`m.add_child(folium.ClickForMarker(popup="Waypoint"))`

Ejemplo4b1.py



- ▶ Utilizando la librería Vega se pueden incluir gráficas:
 - ▶ https://python-visualization.github.io/folium/latest/user_guide/ui_elements/popups.html#Vega-chart-in-popup



► Iconos con rotación:

```
kw = {"prefix": "fa", "color": "green", "icon": "arrow-up"}  
angle = 180  
icon = folium.Icon(angle=angle, **kw)  
folium.Marker(location=[41, -72], icon=icon,  
tooltip=str(angle)).add_to(m)
```

► Se pueden utilizar los iconos de Bootstrap:

- <https://getbootstrap.com/docs/3.3/components/>
- `icon=folium.Icon(color="green", icon="bell")`



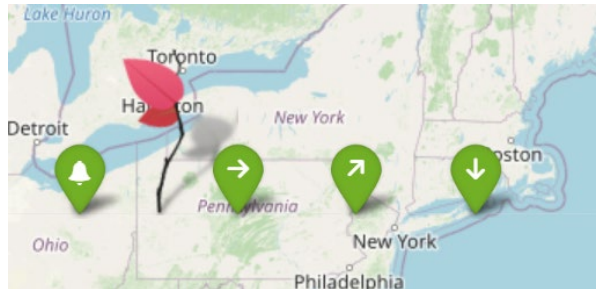
► Iconos personalizados:

```
url = "https://leafletjs.com/examples/custom-  
icons/{}".format
```

```
icon_image = url("leaf-red.png")
```

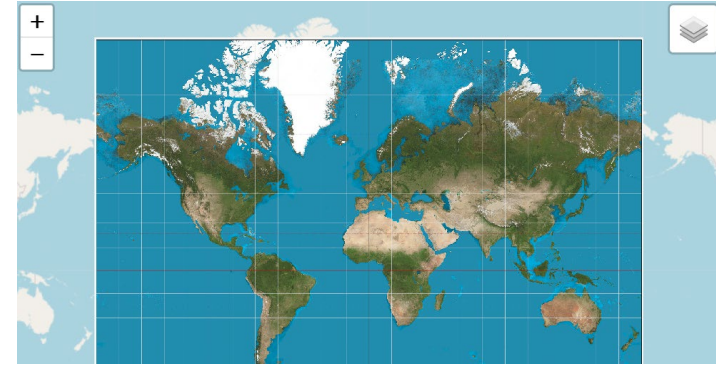
```
shadow_image = url("leaf-shadow.png")
```

```
icon = folium.CustomIcon(icon_image,icon_size=(38,  
95), icon_anchor=(22, 94), shadow_image=shadow_image,  
shadow_size=(50, 64), shadow_anchor=(4, 62),  
popup_anchor=(-3, -76))
```



- Ya hemos visto la capa de teselas, pero además se pueden añadir: Imágenes, Video y WMS

```
folium.raster_layers.ImageOverlay(  
    image="https://upload.wikimedia.org/wikipedia/commons/f/f4/  
    /Mercator_projection_SW.jpg",  
    name="Imagen jpeg",  
    bounds=[[-82, -180], [82, 180]],  
    opacity=1,  
    interactive=False,  
    cross_origin=False,  
    zindex=1,  
    alt="Wikipedia File:Mercator projection SW.jpg",  
).add_to(m)
```



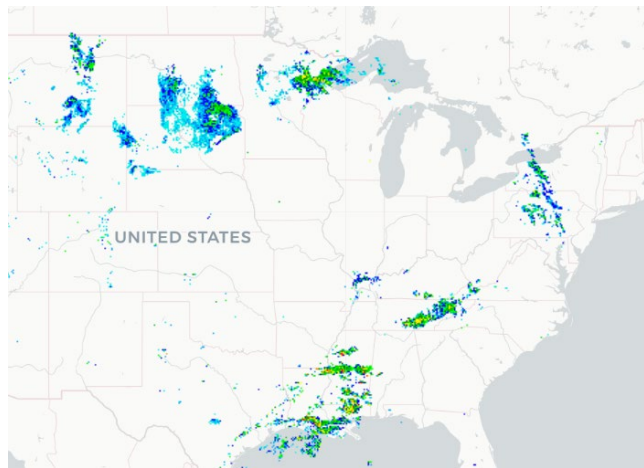
► Capa de Vídeo:

```
video = folium.raster_layers.VideoOverlay(  
    video_url="https://www.mapbox.com/bites/00188/patricia_nasa.webm",  
    bounds=[[32, -130], [13, -100]],  
    opacity=0.65,  
    attr="Video from patricia_nasa",  
    autoplay=True,  
    loop=False,  
)
```



- ▶ Capa WMS (Servicio de mapas Web), mapas de datos dinámicos georreferenciados

```
folium.WmsTileLayer(  
    url="https://mesonet.agron.iastate.edu/cgi-bin/wms/nexrad/n0r.cgi",  
    name="test",  
    fmt="image/png",  
    layers="nexrad-n0r-900913",  
    attr="Weather data © 2012 IEM Nexrad",  
    transparent=True,  
    overlay=True,  
    control=True,  
) .add_to(m)
```



Ejemplo4b2.py

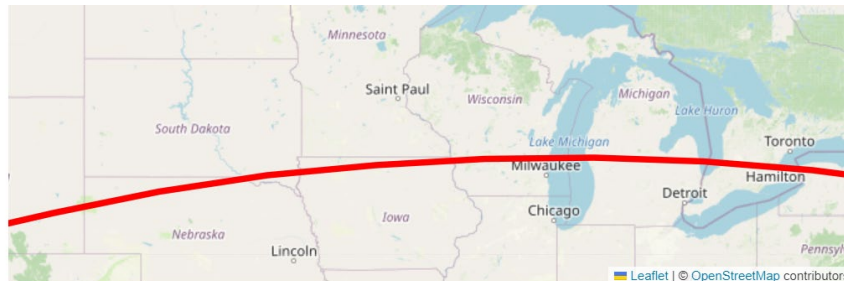


- ▶ Además de las marcas se puede añadir:
 - ▶ Capas vectoriales: polilíneas, rectángulos, polígonos, líneas de colores:
 - ▶ https://python-visualization.github.io/folium/latest/user_guide/vector_layers.html



► Polilineas:

```
coordinates = [  
    [42.3581, -71.0636], [42.82995815, -74.78991444], [43.17929819, -78.56603306],  
    [43.40320216, -82.37774519], [43.49975489, -86.20965845], [43.46811941, -90.04569087],  
    [43.30857071, -93.86961818], [43.02248456, -97.66563267], [42.61228259, -101.41886832],  
    [42.08133868, -105.11585198], [41.4338549, -108.74485069], [40.67471747, -112.29609954],  
    [39.8093434, -115.76190821], [38.84352776, -119.13665678], [37.7833, -122.4167]]  
  
folium.PolyLine(locations=coordinates,color="#FF0000",  
weight=5,tooltip="From Boston to San Francisco").add_to(m)
```



► Rectángulo:

```
kw = {"color": "blue", "line_cap": "round", "fill": True,  
      "fill_color": "red", "weight": 5, "popup": "Tokyo, Japan",  
      "tooltip": "<strong>Click me!</strong>"}
```

```
folium.Rectangle(  
    bounds=[[35.681, 139.766], [35.691, 139.776]],  
    line_join="round", dash_array="5, 5", **kw).add_to(m)
```



► Polígonos:

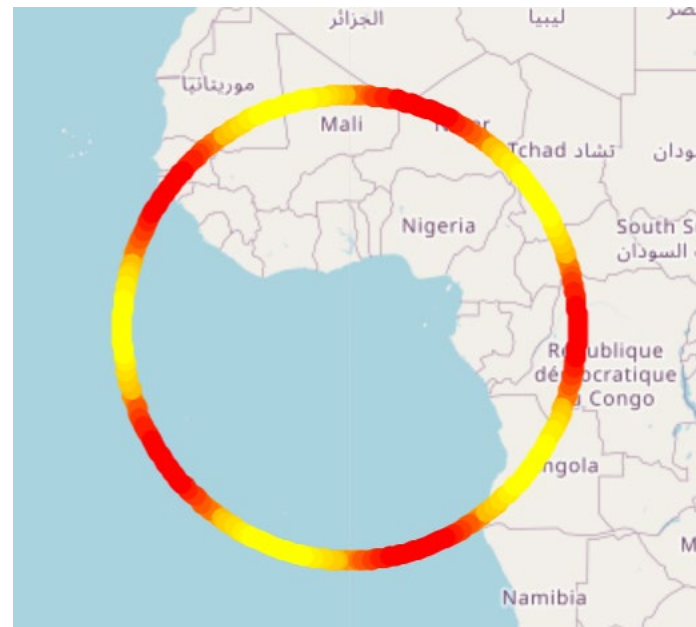
```
locations = [[35.6762, 139.7795], [35.6718, 139.7831],  
[35.6767, 139.7868], [35.6795, 139.7824], [35.6787, 139.7791]]  
folium.Polygon(  
    locations=locations, color="blue", weight=6,  
    fill_color="red", fill_opacity=0.5, fill=True,  
    popup="Tokyo, Japan", tooltip="Click me!").add_to(m)
```



► Líneas de colores:

```
color_line = folium.ColorLine(  
    positions=list(zip(lats, lons)),  
    colors=colors,  
    colormap=["y", "orange", "r"],  
    weight=10,  
).add_to(m)
```

Ejemplo4b3.py



- ▶ GeoJSON: es un formato de fichero que codifica información geográfica, basado en JSON (JavaScript Object Notation)
 - ▶ Soporta los siguientes tipos de geometrías: Point, LineString, Polygon, MultiPoint, MultiLineString y MultiPolygon
- ▶ TopoJSON: es una extensión de GeoJSON que codifica la topología, es decir las relaciones entre las entidades geográficas
 - ▶ Por ejemplo, si representamos las fronteras de los países en un mapa en TopoJSON solo aparecen una vez, esto reduce el tamaño del archivo y mejora la coherencia topológica (los arcos compartidos garantizan que no hay huecos entre polígonos adyacentes)
- ▶ TopoJSON vs GeoJSON: se debe usar TopoJSON para datos con muchas fronteras compartidas y GeoJSON para mayor simplicidad y compatibilidad inmediata



► Ejemplo:

```
folium.GeoJson(antarctic_ice_edge, name="geojson").add_to(m)
```

```
folium.TopoJson(json.loads(requests.get(antarctic_ice_shelf_topo)  
.text),"objects.antarctic_ice_shelf",name="topojson").add_to(m)
```

```
folium.LayerControl().add_to(m)
```

https://python-visualization.github.io/folium/latest/user_guide/geojson/geojson.html



- ▶ Para crear un mapa de coropletas se puede combinar los datos de un DF y las geometrías de un fichero Geo/TopoJSON
- ▶ Los esquemas de color se pueden definir de diferentes formas:

<https://notebook.community/racu10/emapy/notebooks/Colormaps>



► Ejemplo:

```
prov_data = pd.read_csv(prov_paro, encoding='iso-8859-1')
# Filtrar para los datos más recientes y excluir el total nacional
prov_data_filtered = prov_data[(prov_data['Año'] == 2025) &
    (prov_data['Trimestre'] == 4) & (prov_data['codigo'] != 0)].copy()
#El código de provincia en el geojson es str
prov_data_filtered['codigo'] = prov_data_filtered['codigo'].astype(str)
    .zfill(2)

m = folium.Map(location=[40.42, -3.7], zoom_start=8, tiles=..., attr=...)

folium.Choropleth(geo_data=prov_geo,name="choropleth",data=prov_data_filtere
d,columns=["codigo", "Paro"],key_on="properties.codigo",
fill_color="YlGn",fill_opacity=0.4,line_opacity=1.0,legend_name="Tasa de
Paro (%)").add_to(m)
```



Mapas de coropletas



► Tooltip interactivo con múltiples variables

Cargar el geojson y añadir los datos de paro

```
with open(prov_geo, 'r', encoding='utf-8') as f:  
    geojson_data = json.load(f)
```

```
# Crear un diccionario para mapear código -> paro  
paro_dict = dict(zip(prov_data_filtered['codigo'],  
    prov_data_filtered['Paro']))
```

```
# Añadir los datos de paro al geojson  
for feature in geojson_data['features']:  
    codigo = feature['properties']['codigo']  
    paro_value = paro_dict.get(codigo, 'No disponible')  
    feature['properties']['Paro'] = paro_value
```



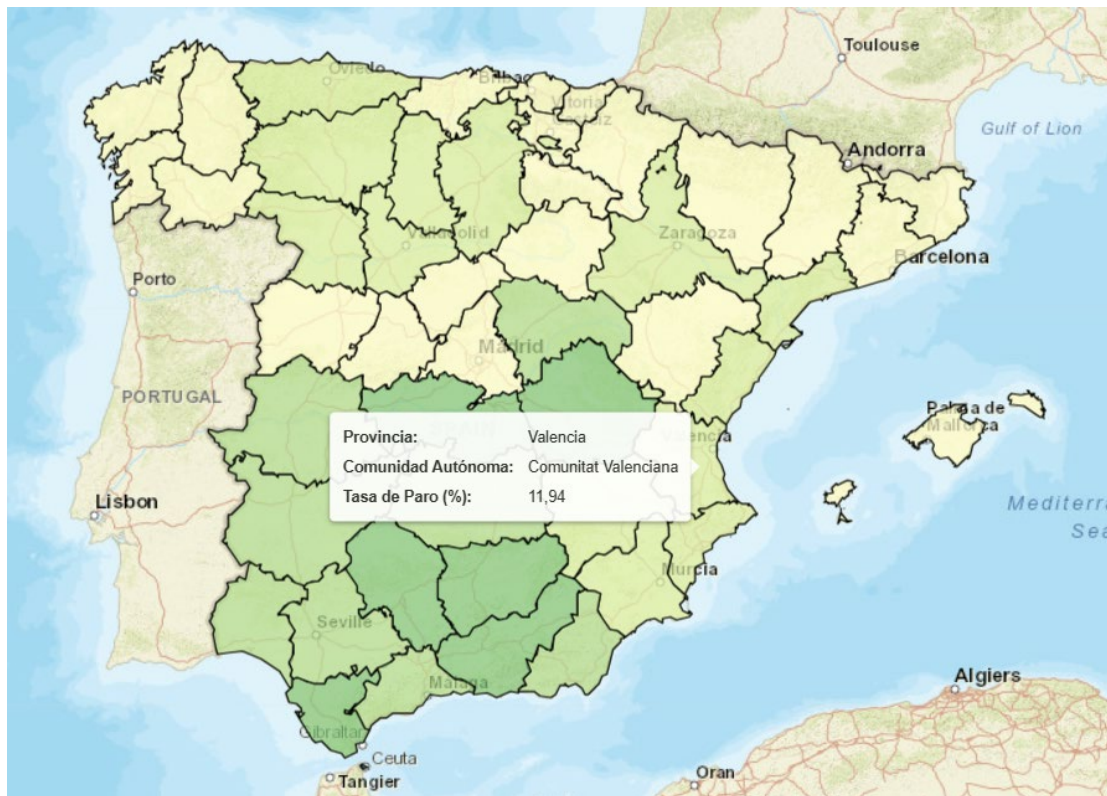
► Tooltip interactivo con múltiples variables

```
choropleth =  
folium.Choropleth(geo_data=geojson_data,name="choropleth"  
,data=prov_data_filtered,columns=["codigo", "Paro"],  
key_on="properties.codigo",fill_color="YlGn",  
fill_opacity=0.7,line_opacity=1.0,  
legend_name="Tasa de Paro (%)").add_to(m)
```

```
folium.GeoJsonTooltip(fields=['texto', 'ccaa', 'Paro'],  
aliases=['Provincia:', 'Comunidad Autónoma:',  
'Tasa de Paro (%)'],  
localize=True).add_to(choropleth.geojson)
```



Mapas de coropletas



- Vamos a crear una capa de marcas a partir de Geojson con las paradas de metro y bus en Valencia:

```
gjson = folium.GeoJson(json.load(open("transporte-barrios.geojson",  
encoding = 'iso-8859-1')), name="geojson",  
marker=folium.vector_layers.CircleMarker(radius=5.0, fill='True',  
fill_color='black',color='red', fill_opacity=1.0)).add_to(m)
```

```
folium.features.GeoJsonPopup(fields=['transporte','stop_name'],labels=False  
) .add_to(gjson)
```

<https://opendata.vlci.valencia.es/dataset/>



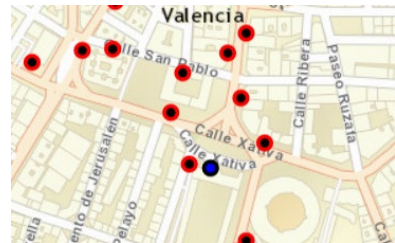
- Podemos utilizar una función de estilo para dibujar diferentes las paradas de metro:

```
def style_function(feature):  
    default_style = {'fillColor': 'black', 'color': 'red'}  
    if feature['properties']['transporte'] == 'metrovlc':  
        default_style['fillColor'] = 'blue'  
        default_style['color'] = 'black'  
    return default_style
```

...

```
marker=folium.vector_layers.CircleMarker(radius=5.0, fill='True',  
fill_opacity=1.0),style_function=style_function...
```

Ejemplo4b4.py



- ▶ Se puede utilizar GeoPandas directamente sin utilizar GeoJson, ya que se convierte en memoria

```
folium.GeoJson(provincias).add_to(m)
```

- ▶ Se puede pasar a GeoJson y aplicar estilo:

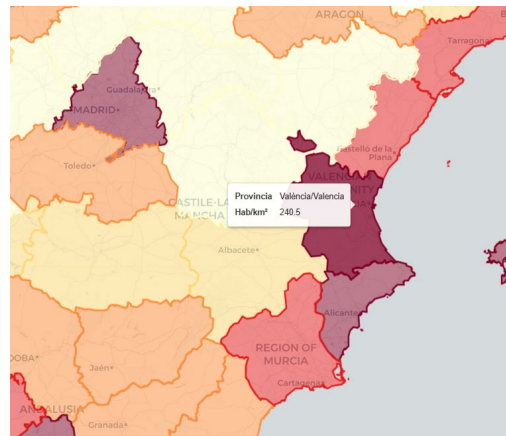
```
geojson = folium.GeoJson(provincias, name="Densidad",  
    style_function=style_densidad,  
    tooltip=folium.GeoJsonTooltip(  
        fields=["NAMEUNIT", "densidad"],  
        aliases=["Provincia", "Hab/km²"], localize=True))
```



- ▶ `GeoDataFrame.explore()` te permite saltarte casi todo el código de Folium

```
m = provincias.explore(  
    column="densidad", cmap="YlOrRd", scheme="quantiles",  
    legend=True, tooltip=["NAMEUNIT", "densidad"],  
    tooltip_kwds={"aliases": ["Provincia", "Hab/km²"]},  
    popup=True, tiles="CartoDB positron")
```

Ejemplo4b5.py



Plugins de Folium

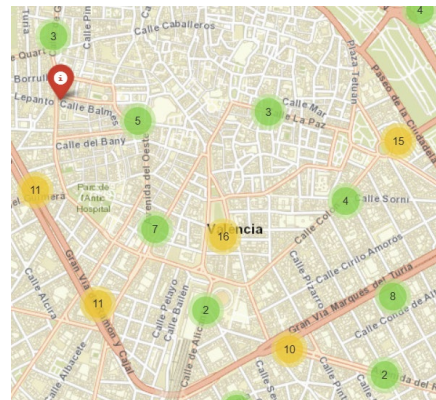


- ▶ MarkerCluster: Agrupa marcadores cercanos en clusters con contador, evitando saturación visual cuando hay muchos puntos (p. ej., todas las paradas de transporte de Valencia); al hacer zoom, los clusters se desagregan automáticamente

```
from folium import plugins
```

```
marker_cluster = plugins.MarkerCluster().add_to(m)
```

```
folium.Marker(...).add_to(marker_cluster)
```



- ▶ MiniMap: Añade mapa de referencia en miniatura en esquina, facilitando navegación en mapas con mucho zoom

```
from folium import
```

```
plugins minimap = plugins.Minimap(toggle_display=True)
```

```
m.add_child(minimap)
```



- ▶ Fullscreen: Botón para expandir el mapa a pantalla completa, mejorando la experiencia de usuario en presentaciones o dashboards

```
plugins.Fullscreen(position='topright').add_to(m)
```

- ▶ Draw: Permite dibujar interactivamente polígonos, líneas y marcadores sobre el mapa, útil para delimitar áreas de interés o diseñar rutas

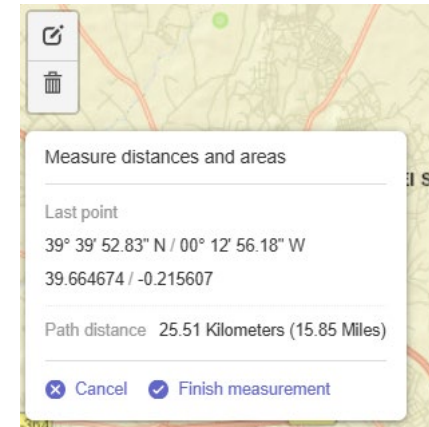
```
plugins.Draw(export=True).add_to(m)
```



- ▶ **MeasureControl**: Herramienta para medir distancias y áreas directamente en el mapa, muy útil para análisis de rutas y zonas de influencia

```
plugins.MeasureControl(position='topleft',  
primary_length_unit='kilometers').add_to(m)
```

Ejemplo4b6.py



- ▶ GPX es un formato de fichero que almacena datos GPS en formato GPS Exchange, que es un estándar abierto utilizado por muchos programas GPS
- ▶ Contiene datos de ubicación de longitud y latitud que pueden incluir waypoints, rutas y entrenamientos
- ▶ Los archivos GPX se guardan en formato XML, lo que permite que múltiples programas y servicios web importen y lean más fácilmente los datos GPS
- ▶ Se utiliza en los principales dispositivos deportivos para exportar actividades de bicicleta y carrera
- ▶ Entre otros se utiliza en: Google Earth, Google Maps, Strava, Garmin Connect,...



- ▶ Además de los datos de longitud, latitud y tiempo, cada punto puede incluir otros datos:
 - ▶ Elevación, Ritmo cardíaco, Cadencia, Temperatura, etc.
- ▶ La librería gpxpy permite leer estos ficheros
- ▶ Para visualizarlos seguiremos estos pasos:
 - ▶ Convertir fichero GPX en DF
 - ▶ Crear mapa base de Folium
 - ▶ Introducir marcas en el mapa
 - ▶ Añadir todo al mapa base
- ▶ Una posibilidad interesante consiste en crear mapas de calor



- ▶ Sin embargo, los HeatMaps de Folium (LeafLet) no funcionan correctamente con pesos:

```
# Normalizas la FC
```

```
df2['HRN'] = (df2["HR"] - df2["HR"].min()) /  
(df2["HR"].max() - df2["HR"].min())
```

```
# Creas los datos geolocalizados
```

```
heat_data = df2[["Latitude", "Longitude",  
"HRN"]].values.tolist()
```

```
HeatMap(heat_data, radius=15, blur=1, min_opacity=0.25,  
gradient={0.2: "blue", 0.4: "lime", 0.6: "yellow", 0.8:  
"orange", 1.0: "red"}).add_to(hmap)
```

```
Ejemplo4b7.py
```



- ▶ Lo vamos a hacer con ColorLine
- ▶ El color debe ir en función de un parámetro

```
cm = br.LinearColormap(colors=['green','yellow','orange',  
'red'], index=[0.0,0.35,0.8,1.0], vmin=0.0, vmax=1.0)
```

```
line=folium.features.ColorLine(list(zip(df2.Latitude,df2.  
Longitude)),colors = df2['HRN'].tolist(), colormap = cm,  
weight=6)
```

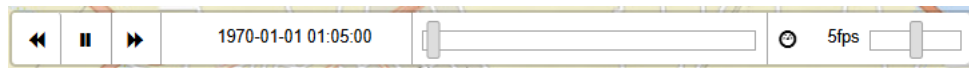
```
hmap.add_child(line)
```

Ejemplo4b8.py



► Animación en mapas: TimestampedGeoJson

```
features = []  
for idx, row in df2.iterrows():  
    features.append({'type': 'Feature',  
                    'geometry': {'type': 'Point',  
                                'coordinates': [row.Longitude, row.Latitude]},  
                    'properties': {'time': row['Timestamp'].isoformat(),  
                                'popup': f"HR : {row['HR']}", 'icon': 'circle',  
                                'iconstyle': {'fillColor': 'red', 'radius': 2}}})  
  
plugins.TimestampedGeoJson({'type': 'FeatureCollection', 'features':  
features}, period='PT15S', add_last_point=True).add_to(hmap)  
Ejemplo4b9.py
```



▶ Ejercicio:

- ▶ Utilizando el fichero MaratonVLC17.gpx crea un nuevo DF:
 - ▶ Selecciona un punto cada 15 segundos (resample)
 - ▶ Calcula la velocidad en cada punto
 - ▶ Calcula el máximo y mínimo de la velocidad
 - ▶ Dibuja la ruta mediante una línea coloreando en función de la velocidad
 - ▶ Incluye una marca cada 5 kms (kms17.csv)
 - ▶ Incluye una marca cada 5 minutos recorridos que incluya:
 - ▶ Ritmo cardíaco en el punto
 - ▶ Velocidad media de los últimos 10 puntos (m/s)
 - ▶ Ritmo medio de los últimos 10 puntos en Minutos/km



- ▶ <https://opendata.vlci.valencia.es/dataset/>
- ▶ <https://www.ine.es/>
- ▶ <https://datos.gob.es/es/catalogo>
- ▶ <https://opendata.esri.es/>
- ▶ <https://ourworldindata.org/>
- ▶ **Plugins Folium:**
https://python-visualization.github.io/folium/latest/user_guide/plugins.html

